Gerência de configuração

Introdução ao terraform

Prof. Diego Cardoso Borda Castro

Introdução

- Formas de criar uma infraestrutura em cloud
  - Painel web
    - Replicação em diferentes ambientes
    - Processo manual
    - Problemas de automatizar
  - Ferramentas de CLI
    - Azure CLI, AWS CLIS, …
    - Execução de scripts
    - Mais automação
    - Replicação mais eficiente
    - Ordem de execução
    - Documentação

Introdução

- Formas de criar uma infraestrutura em cloud
  - Linguagem de programação
    - Azure SDK, AWS SDK, …
    - Imperativa
  - Ferramentas de Infraestrutura como código
    - Declarativa
      - Não importa a ordem de execução
      - Declara cada elemento que deve ser provisionado na sua infra
        - Subrede com range de ip, VMs, …

Introdução

O que é IaC?

- Gerenciamento de infra através de arquivos de configuração
- Versionamento (Git) e rastreabilidade
- Ambientes repetíveis e confiáveis
- Eliminação de processos manuais

Por que Terraform?

- Multi-cloud: Uma vez criado, serve para qualquer nuvem
- Linguagem Declarativa (HCL)
- Gerenciamento de estado (Terraform State)
- Ecossistema robusto de Providers

Terraform e seus Concorrentes

O Terraform destaca-se pela sua natureza **declarativa** e agnóstica a provedor.

Concorrentes Principais: Pulumi (Imperativo), AWS CloudFormation, Azure ARM Templates e Google Deployment Manager.

Introdução

Definição

- Ferramenta de **Infraestrutura como Código (IaC)**
- Criada pela **HashiCorp**
- **Objetivo:** Automatizar infraestrutura usando código

O que ele permite criar?

Servidores, Redes, Bancos de dados, Containers, Clusters Kubernetes, Regras de firewall, DNS e diversos Cloud Providers.

Vantagens:

- Versionado e Reproduzível
- Automatizado e Auditável
- Escalável

Infra sem IaC

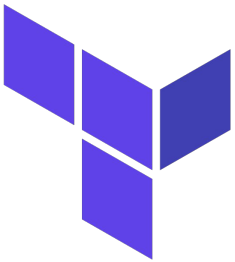
- acessa a cloud
- cria VM
- cria firewall
- instala software
- configura manualmente

Introdução

- Terraform
  - Provisionamento de infra
  - Está relacionado a criação dos recursos
- Ansible
  - Gerenciamento de configuração
  - Está relacionado a configuração de recursos já provisionados

Não vão competir e sim se complementar

Introdução



Terraform

- Linguagem HCL (Declarativa)
- Amplo ecossistema de providers
- Gerenciamento de Estado (.tfstate)
- Foco em Provisionamento



Pulumi

- Linguagens reais (Python, JS, Go)
- Abstrações poderosas (SDKs)
- Integração nativa com IDEs
- Forte para Desenvolvedores

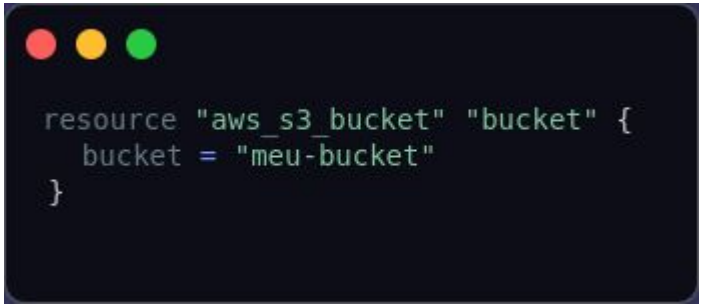


Crossplane

- Extensão do Plano de Controle K8s
- Infra como Recursos do K8s
- Drift Correction contínuo
- Ideal para Plataformas Internas

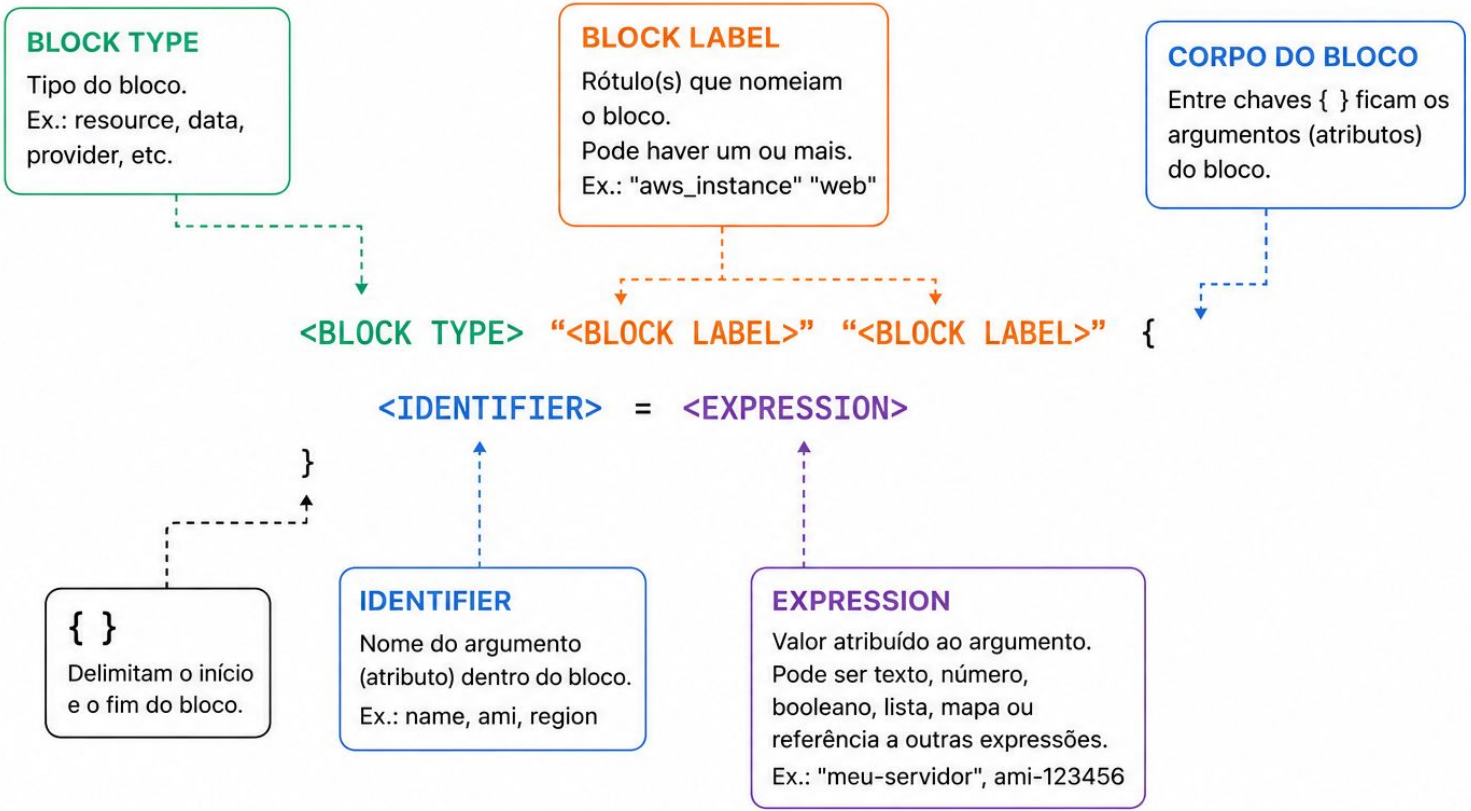
Terraform

- Modelo declarativo
  - Você diz: "o que deseja"
  - Terraform decide: "como criar"
  - Você não explicar ordem
    - Terraform faz automaticamente
      - cria
      - atualiza
      - destrói



Terraform

- Hashicorp Configuration Language
  - Funciona em blocos
  - Você vai criar vários blocos e depois aplica no terraform
  - Estrutura
    - Tipo de bloco
      - output, resource, datasource …
    - Label
      - Nome dele no projeto para referenciar
    - Identificador e expressão
      - Configurações do seu bloco


BLOCK TYPE
Tipo do bloco.
Ex.: resource, data, provider, etc.
BLOCK LABEL
Rótulo(s) que nomeiam o bloco.
Pode haver um ou mais.
Ex.: "aws_instance" "web"
CORPO DO BLOCO
Entre chaves { } ficam os argumentos (atributos) do bloco.
<BLOCK TYPE> "<BLOCK LABEL>" "<BLOCK LABEL>" {
<IDENTIFIER> = <EXPRESSION>
}
{ }
Delimitam o início e o fim do bloco.
IDENTIFIER
Nome do argumento (atributo) dentro do bloco.
Ex.: name, ami, region
EXPRESSION
Valor atribuído ao argumento.
Pode ser texto, número, booleano, lista, mapa ou referência a outras expressões.
Ex.: "meu-servidor", ami-123456

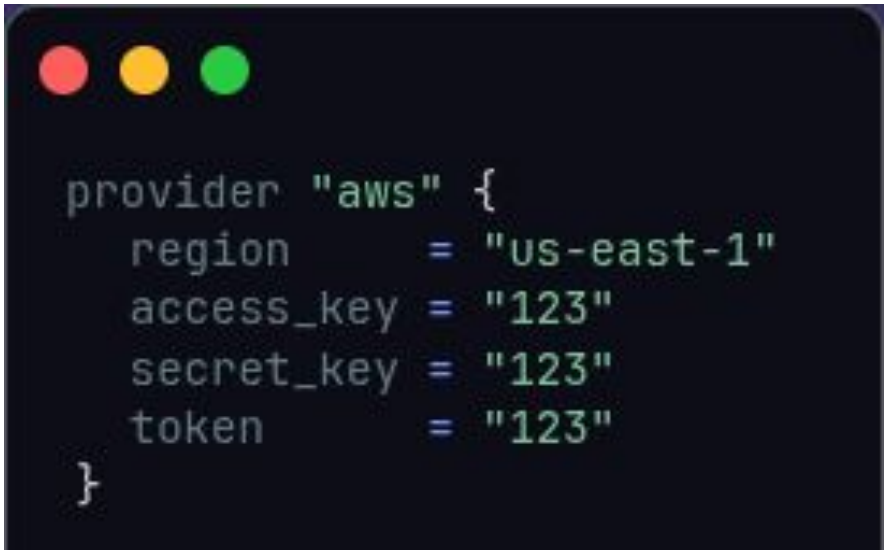
Terraform (Blocos)

- Terraform settings
  - Arquivo .tf
  - Terraform registry



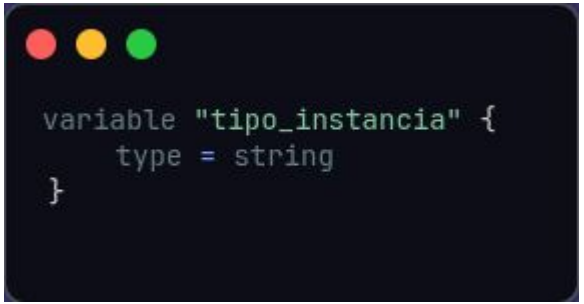
Terraform (Blocos)

- Provider
  - Pode ter vários
  - Multicloud
- Não se deve colocar vars no arquivo
  - Pouco seguro



Terraform (Blocos)

- Credential Provider Chain
- Variáveis
  - TF_VAR_tipo_instancia
    - export TF_VAR_tipo_instancia="t3.medium"



Terraform (Blocos)

- Bloco mais utilizado
  - Declara o recurso a ser criado
  - Elemento a ser criado
- Argumentos
- Bloco Aninhado

```
resource "azurerm_linux_virtual_machine" "maquina_labs_tf" {
  name                = "maquina-labs-tf"
  resource_group_name = "meu_grupo_de_recursos"
  location            = "East US"
  size                = "Standard_B1s"
  admin_username      = "adminuser"

  network_interface_ids = [
    azurerm_network_interface.minha_placa_rede.id,
  ]

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-focal"
    sku       = "20_04-lts"
    version   = "latest"
  }
}
```

Terraform (Blocos)

- Variável
  - Crie configurações de variáveis para deixar seu projeto reutilizável
  - Posso usar o mesmo terraform mudando só a imagem
    - terraform.tfvars
    - terraform apply -var="regiao=East US"

```
variable "regiao" {
  type        = string
  default     = "Brazil South"
  description = "Região da Azure"
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "rg" {
  name     = "rg-exemplo"
  location = var.regiao
}

resource "azurerm_virtual_network" "vnet" {
  name                = "vnet-exemplo"
  location            = var.regiao
  resource_group_name = azurerm_resource_group.rg.name

  address_space = ["10.0.0.0/16"]
}
```

Terraform (Blocos)

- Locals
  - Cria variáveis internas reutilizáveis dentro do código
  - Var Interna



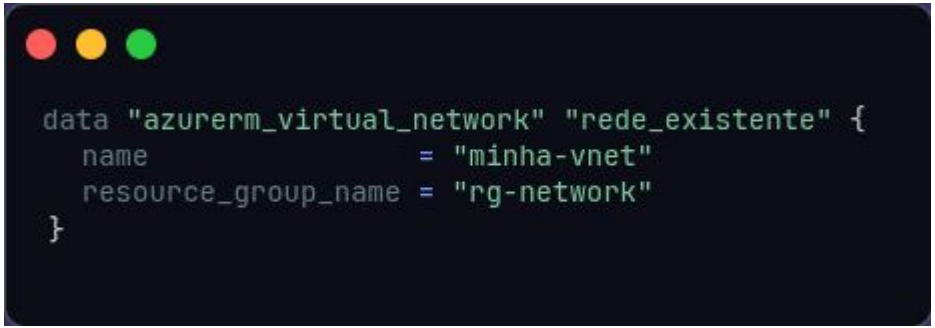
```
locals {
  nome_empresa = "cefet"
  ambiente     = "dev"

  nome_rg = "${local.nome_empresa}-${local.ambiente}-rg"
}

resource "azurerm_resource_group" "rg" {
  name     = local.nome_rg
  location = "Brazil South"
}
```

Terraform (Blocos)

- Data source
  - Vai linkar com um elemento já criado no seu cloud provider
  - Poder linkar a rede já criada anteriormente



```
data "azurerm_virtual_network" "rede_existente" {
  name                = "minha-vnet"
  resource_group_name = "rg-network"
}
```

Terraform (Blocos)

- Output
  - Serve para pegar informações geradas pela automação
  - Ex.: pegar o ip da máquina criada sem abrir o cloud provider



```
output "nome_resource_group" {
  value = azurerm_resource_group.rg.name
}

output "localizacao" {
  value = azurerm_resource_group.rg.location
}

output "id_vnet" {
  value = azurerm_virtual_network.vnet.id
}

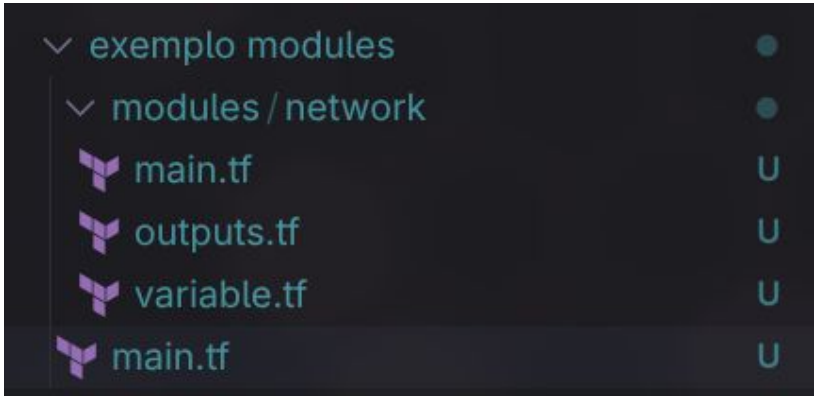
output "enderecos_vnet" {
  value = azurerm_virtual_network.vnet.address_space
}
```

Terraform (Blocos)

- Modules
  - Serve para criar partes reutilizáveis
  - Importar vários arquivos

```
module "rede" {
  source = "./modules/network"

  nome_rg = "rg-exemplo"
  regiao  = "Brazil South"
}
```



Bloco	Função
terraform	Configura o Terraform e providers
provider	Define conexão com cloud/provider
variable	Entrada de dados
resource	Cria recursos
data	Consulta e linka recursos existentes
output	Exibe informações após execução
locals	Valores reutilizáveis internos
module	Reutilização de infraestrutura

Nosso primeiro exemplo

- yay -S azure-cli
- ou
- curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash

- sudo pacman -S terraform
- ou
- sudo apt install terraform

Nosso primeiro exemplo

- az login

- terraform version
- az account show
- az account list --output table

- terraform init
  - Baixa providers/plugins
  - Inicia o projeto

```
~ ❱ terraform version
Terraform v1.15.1
on linux_amd64
```

```
…/terraform/exemplo 1 main ? ❱ terraform init
Initializing provider plugins found in the configuration...
- Finding hashicorp/azurerm versions matching "~> 3.0"...
- Installing hashicorp/azurerm v3.117.1...
- Installed hashicorp/azurerm v3.117.1 (signed by HashiCorp)

Initializing the backend...


Terraform has created a lock file .terraform.lock.hcl to record the p
selections it made above. Include this file in your version control r
so that Terraform can guarantee to make the same selections by defaul
you run "terraform init" in the future.

Terraform has been successfully initialized!
```

Nosso primeiro exemplo

- Apply
  - Executa mudanças
- Destroy
  - Remove recursos

```
  Enter a value: yes

azurerm_linux_virtual_machine.vm: Creating...
azurerm_linux_virtual_machine.vm: Still creating... [00m10s elapsed]
azurerm_linux_virtual_machine.vm: Still creating... [00m20s elapsed]
azurerm_linux_virtual_machine.vm: Still creating... [00m30s elapsed]
azurerm_linux_virtual_machine.vm: Still creating... [00m40s elapsed]
azurerm_linux_virtual_machine.vm: Still creating... [00m50s elapsed]
azurerm_linux_virtual_machine.vm: Creation complete after 51s [id=/subsc
ups/rg-aula1/providers/Microsoft.Compute/virtualMachines/vm-aula1]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

ip_publico_vm = "74.249.189.102"

…/terraform/exemplo 1 main ? ❯
```

```
…/terraform/exemplo 1 main ? ❯ terraform apply -var="vm_size_index=2"
azurerm_resource_group.rg: Refreshing state... [id=/subscriptions/1f1ce909-7e19
-4a46-9206-0451240535e7/resourceGroups/rg-aula1]
azurerm_public_ip.ip: Refreshing state... [id=/subscriptions/1f1ce909-7e19-4a46
-9206-0451240535e7/resourceGroups/rg-aula1/providers/Microsoft.Network/publicIP
Addresses/ip-aula1]
azurerm_virtual_network.vnet: Refreshing state... [id=/subscriptions/1f1ce909-7
e19-4a46-9206-0451240535e7/resourceGroups/rg-aula1/providers/Microsoft.Network/
virtualNetworks/vnet-aula1]
azurerm_subnet.subnet: Refreshing state... [id=/subscriptions/1f1ce909-7e19-4a4
6-9206-0451240535e7/resourceGroups/rg-aula1/providers/Microsoft.Network/virtual
Networks/vnet-aula1/subnets/subnet-aula1]
azurerm_network_interface.nic: Refreshing state... [id=/subscriptions/1f1ce909-
7e19-4a46-9206-0451240535e7/resourceGroups/rg-aula1/providers/Microsoft.Network
/networkInterfaces/nic-aula1]
```

Nosso primeiro exemplo

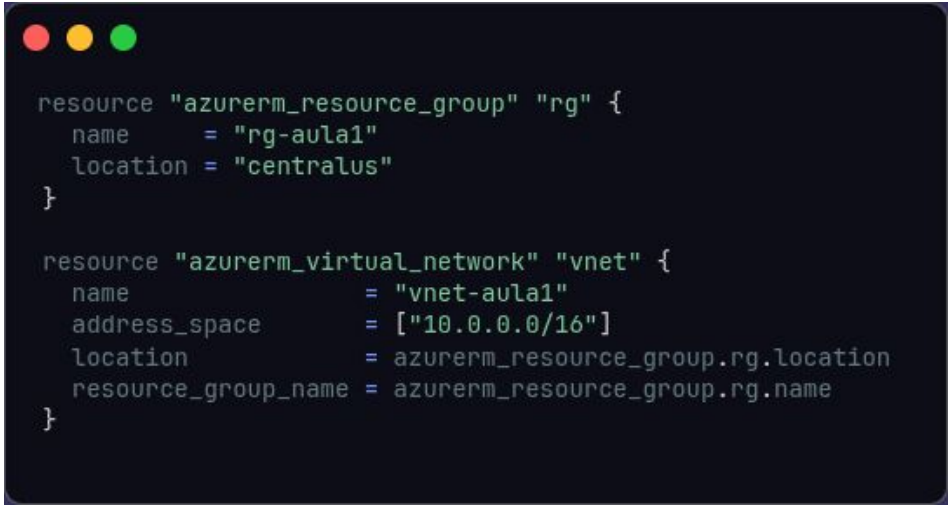
- Provider Oficial
  - Usa o plugin da Azure (azurerm)
- Versão Fixa (=3.99.0)
- features {}
  - Bloco obrigatório para o Terraform iniciar a conexão com a nuvem

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "=3.99.0"
    }
  }
}

provider "azurerm" {
  features {}
}
```

Nosso primeiro exemplo

- Cria o contêiner lógico
  - Região centralus
  - Agrupar e organizar os recursos.
- Rede Virtual (vnet)
  - Cria a rede privada
- Vínculo Dinâmico
  - A rede puxa automaticamente o local e o nome do grupo de recursos



```
resource "azurerm_resource_group" "rg" {
  name     = "rg-aula1"
  location = "centralus"
}

resource "azurerm_virtual_network" "vnet" {
  name                = "vnet-aula1"
  address_space       = ["10.0.0.0/16"]
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
}
```

Nosso primeiro exemplo

- Cria Sub-rede
- Divide a rede principal em um pedaço menor
  - organizar e isolar as máquinas
- Vínculo Automático
- IP Fixo (Static)
- Standard
  - Define a categoria do IP

```
resource "azurerm_subnet" "subnet" {
  name                 = "subnet-aula1"
  resource_group_name  = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = ["10.0.1.0/24"]
}

resource "azurerm_public_ip" "ip" {
  name                = "ip-aula1"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  allocation_method   = "Static"
  sku                 = "Standard"
}
```

Nosso primeiro exemplo

- nic
  - Interface Virtual
- IP Privado (Dynamic)
  - Pega um endereço IP interno automaticamente
- Uso de variável

```
resource "azurerm_network_interface" "nic" {
  name                = "nic-aula1"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.subnet.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id          = azurerm_public_ip.ip.id
  }
}

variable "vm_sizes" {
  default = [
    "Standard_B2pts_v2",
    "Standard_A2m_v2",
    "Standard_A4_v2",
    "Standard_B2ms",
  ]
}
```

- Criação da VM
  - Crio uma SSH da minha máquina para a vm
  - Definição da imagem da máquina
  - Output

```
output "ip_publico_vm" {
  value = azurerm_public_ip.ip.ip_address
}
```

```
variable "vm_size_index" {
  default = 0
}

resource "azurerm_linux_virtual_machine" "vm" {
  name                = "vm-aula1"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location

  size = var.vm_sizes[var.vm_size_index]

  admin_username = "azureuser"

  disable_password_authentication = true

  network_interface_ids = [
    azurerm_network_interface.nic.id
  ]

  admin_ssh_key {
    username   = "azureuser"
    public_key = file("/home/diego/.ssh/id_rsa.pub")
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts-arm64"
    version   = "latest"
  }
}
```